

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Justifier le choix de Python pour l'enseignement de la programmation
- Installer et configurer un environnement Python (IDLE, VS Code, Thonny)
- Déclarer et utiliser les types de base (int, float, str, bool)
- Structurer un programme avec conditions (if/elif/else) et boucles (for/while)
- Définir et appeler des fonctions avec paramètres et valeur de retour
- Lire et écrire dans un fichier texte depuis un script Python

Plan du cours

1. Pourquoi Python pour l'enseignement — 10 min
2. Environnements de développement — 10 min
3. Variables et types — 15 min
4. Conditions (if/elif/else) — 15 min
5. Boucles (for, while) — 15 min
6. Fonctions (def, return, paramètres) — 15 min
7. Entrées/sorties et fichiers — 10 min
8. QCM et conclusion — 5 min

1. Pourquoi Python pour l'enseignement ? (10 min)

Python est devenu le langage de programmation le plus utilisé dans l'éducation et la recherche.

```
graph LR
  subgraph "Pseudo-code algorithmique"
    PC["SI note >= 10<br/> AFFICHER 'Validé'<br/>SINON<br/> AFFICHER 'Non validé'"]
  end
  subgraph "Code Python"
    PY["if note >= 10:<br/> print('Validé')<br/>else:<br/> print('Non validé')"]
  end
  subgraph "Exécution"
    EX["Entrée: 15<br/>Sortie: Validé"]
  end
  PC -->|"Traduction naturelle"| PY
  PY -->|"Interprétation"| EX
  style PC fill:#f39c12,color:#fff
  style PY fill:#3498db,color:#fff
```

```
style EX fill:#2ecc71,color:#fff
```

Atouts pédagogiques : syntaxe lisible (proche du pseudo-code), typage dynamique, gratuit/open source, bibliothèques riches, communauté éducative active. Python est le langage officiel du programme NSI en France.

Utilisations en contexte scolaire :

- Mathématiques : calculs, simulation, tracés (matplotlib)
- Langues : analyse de textes, quiz de vocabulaire
- Sciences : traitement de données, graphiques
- SNT/NSI : programme officiel — langage de référence

2. Environnements de développement (10 min)

Outil	Type	Public cible	Avantage
IDLE	IDE intégré	Débutants	Installé avec Python
Thonny	IDE éducatif	Collège/Lycée	Débogueur visuel, idéal pour découvrir
VS Code	Éditeur pro	Lycée/Université	Extensions, terminal intégré
Replit	En ligne	Tous	Aucune installation
Google Colab	Notebook Web	Recherche/NSI	Tutoriels partagés

Recommandation : Thonny pour débiter, puis VS Code pour les projets.

```
# Vérifier Python
python --version
# Exécuter un script
echo 'print("Bonjour ENS !")' > hello.py && python hello.py
```

3. Variables et types (15 min)

En Python, les variables sont des étiquettes pointant vers des objets. Pas de déclaration de type explicite.

```
age = 20 # int
pi = 3.14159 # float
nom = "Madani" # str
est_enseignant = True # bool

print(type(age)) # <class 'int'>
print(f"{nom} a {age} ans") # f-string (Python 3.6+)

# Conversion de type
age_str = "20"
age_int = int(age_str) # "20" → 20
note = float(input("Note : ")) # Saisie → float
```

Parallèle pseudo-code ↔ Python :

Concept	Pseudo-code	Python
Affectation	age ← 20	age = 20

Affichage	<code>AFFICHER age</code>	<code>print(age)</code>
Saisie	<code>LIRE age</code>	<code>input()</code>
Commentaire	<code>// texte</code>	<code># texte</code>
Comparaison	<code>age == 18</code>	<code>age == 18</code>

Piège : = est l'affectation, == est la comparaison.

4. Conditions (15 min)

```
graph TD
DEBUT(["Début"]) --> SAISIE[Saisir note]
SAISIE --> COND{"note >= 10 ?"}
COND -->|Oui| VALIDE[Afficher 'Validé']
COND -->|Non| NONVALIDE[Afficher 'Non validé']
VALIDE --> FIN(["Fin"])
NONVALIDE --> FIN
style DEBUT fill:#2ecc71,color:#fff
style FIN fill:#e74c3c,color:#fff
style COND fill:#f39c12,color:#fff

note = float(input("Note sur 20 : "))
if note >= 16:
    print("Très bien")
elif note >= 14:
    print("Bien")
elif note >= 12:
    print("Assez bien")
elif note >= 10:
    print("Passable")
else:
    print("Non validé")
```

Opérateurs : ==, !=, <, >, <=, >=, and, or, not

Règles d'or :

- Ne pas oublier les : après if, elif, else
- Indentation cohérente (4 espaces)
- `float(input())` pour les nombres décimaux

5. Boucles (15 min)

Boucle for — parcours d'une séquence

```
for i in range(5):
    print(f"Itération {i}")

for i in range(1, 10, 2):
    print(i) # 1, 3, 5, 7, 9

noms = ["Ali", "Sara", "Mohamed"]
for nom in noms:
    print(f"Bonjour {nom}")

for i, nom in enumerate(noms):
    print(f"{i+1}. {nom}")
```

Boucle while — répétition conditionnelle

```

reponse = ""
while reponse != "oui" and reponse != "non":
    reponse = input("Continue ? (oui/non) : ").lower()

compteur = 0
while compteur < 5:
    print(f"Compteur : {compteur}")
    compteur += 1

graph LR
    subgraph "Boucle for"
        F1["for i in range(5):"]
        F2["print(i)"]
        F3["i: 0 → 1 → 2 → 3 → 4"]
    end
    subgraph "Boucle while"
        W1["while condition:"]
        W2["instruction"]
        W3["Condition vérifiée<br/>à chaque itération"]
    end
    F1 --> F2 --> F3
    W1 --> W2 --> W3 --> W1
    style F1 fill:#3498db,color:#fff
    style W1 fill:#e74c3c,color:#fff

```

Quand utiliser quoi ? for quand le nombre d'itérations est connu (parcours de séquence).
while quand on attend une condition.

6. Fonctions (15 min)

```

def calculer_moyenne(notes):
    """Calcule la moyenne d'une liste de notes."""
    return sum(notes) / len(notes)

def saluer(nom, message="Bienvenue"):
    print(f"{message}, {nom} !")

notes = [15, 12, 18, 10, 14]
moy = calculer_moyenne(notes)
print(f"Moyenne : {moy:.2f}")

saluer("Ali") # Bienvenue, Ali !
saluer("Sara", "Bonjour") # Bonjour, Sara !

def notes_min_max(notes):
    return min(notes), max(notes) # Retourne un tuple

mini, maxi = notes_min_max(notes)
print(f"Min: {mini}, Max: {maxi}")

```

Pourquoi des fonctions ? Réutilisabilité, lisibilité, testabilité, modularité.

Bonnes pratiques : noms en verbes, docstring entre """ , un retour par fonction.

7. Entrées/sorties et fichiers (10 min)

```

# Saisie utilisateur
nom = input("Entrez votre nom : ")
age = int(input("Entrez votre âge : "))

```

```
# Écriture fichier
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Nom,Note\n")
    f.write("Ali,15\n")
    f.write("Sara,18\n")
# Le with ferme automatiquement

# Lecture fichier
with open("notes.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        print(ligne.strip())

# Modes : "r" lecture, "w" écriture (écrase), "a" ajout

# Traitement CSV simple
with open("notes.txt", "r") as f:
    f.readline() # Ignorer en-tête
    total = 0
    n = 0
    for ligne in f:
        nom, note = ligne.strip().split(",")
        total += float(note)
        n += 1
    print(f"Moyenne : {total/n:.2f}")
```

Contexte pédagogique : le format CSV est omniprésent dans l'éducation (export Moodle, Excel, bulletins).

8. Mini-projet (5 min)

Quiz de vocabulaire anglais/français — combinant tous les concepts :

```
questions = {
    "chat": "cat", "chien": "dog", "maison": "house",
    "livre": "book", "école": "school", "professeur": "teacher"
}

def poser_question(mot_fr, mot_en):
    reponse = input(f"Traduire '{mot_fr}' : ").strip().lower()
    if reponse == mot_en:
        print("✓ Correct !")
        return True
    print(f"✗ Faux. Réponse : {mot_en}")
    return False

score = 0
for fr, en in questions.items():
    if poser_question(fr, en):
        score += 1
print(f"Score : {score}/{len(questions)}")
```

QCM d'auto-évaluation

- Quelle fonction affiche du texte dans la console en Python ?
a) input() b) print() ✓ c) write()
- Quel type retourne input() ?
a) int b) float c) str ✓

3. Quel mot-clé définit une fonction ?
a) function b) def ✓ c) define
4. Quelle boucle est idéale pour parcourir une liste ?
a) while b) for ✓ c) repeat
5. Quel mode d'ouverture permet l'écriture (écrase) ?
a) "r" b) "w" ✓ c) "a"

Conclusion (5 min)

Python est idéal pour l'enseignement : syntaxe claire, concepts universels (variables, conditions, boucles, fonctions), applications concrètes (traitement de fichiers, quiz). La manipulation de CSV prépare les futurs enseignants à traiter les données pédagogiques.

Prochain cours : Projet Python — générateur de fiches de révision et calculateur de moyenne.

Note pédagogique ENS

Transposition : SNT Seconde (Thonny, turtle) / NSI Première (types, fonctions, fichiers) / Mathématiques (équations, simulation).

Démarche : pseudo-code → Python → exécution → variation des paramètres.

Éviter : utiliser Thonny portable ou Replit pour les premières séances.

Bibliographie

- Downey — Think Python (2nd ed., O'Reilly)
- Documentation Python 3 : docs.python.org
- Python pour l'enseignement (Eduscol)